

Generator

Generator

Generator，即数据生成器。当数据很大，手造会累死的时候，我们就需要它来帮助我们自动造数据。

简单的例子

生成两个 范围内的整数：

```
1 // clang-format off
2
3 #include "testlib.h"
4 #include <iostream>
5
6 using namespace std;
7
8 int main(int argc, char* argv[]) {
9     registerGen(argc, argv, 1);
10    int n = atoi(argv[1]);
11    cout << rnd.next(1, n) << " ";
12    cout << rnd.next(1, n) << endl;
13 }
```

为什么要使用 Testlib?

有人说写 generator 不需要用 Testlib，它在这没什么用。实际上这是个不正确的想法。一个好的 generator 应该满足这一点：**在任何环境下对于相同输入它给出相同输出**。写 generator 就避免不了生成随机值，平时我们用的 `rand()` 或 C++11 的 `mt19937/uniform_int_distribution`，当操作系统不同、使用不同编译器编译、不同时间运行等，它们的输出都可能不同（对于非常常用的 `srand(time(nullptr))`，这是显然的），而这就会给生成数据带来不确定性。

需要注意的是，一旦使用了 Testlib，就不能再使用标准库中的 `srand()`，`rand()` 等随机数函数，否则在编译时会报错。因此，**请确保所有与随机相关的函数均使用 Testlib 而非标准库提供的。**

而 Testlib 中的随机值生成函数则保证了相同调用会输出相同值，与 generator 本身或平台均无关。另外。它给生成各种要求的随机值提供了很大便利，如 `rnd.next("[a-z]{1,10}")` 会生成一个长度在 范围内的串，每个字符为 `a` 到 `z`，很方便吧！

Testlib 能做什么？

在一切之前，先执行 `registerGen(argc, argv, 1)` 初始化 Testlib（其中 `1` 是使用的 generator 版本，通常保持不变），然后我们就可以使用 `rnd` 对象来生成随机值。随机数种子取自命令行参数的哈希值，对于某 generator `g.cpp`，`g 100 (Unix-Like)` 和 `g.exe "100" (Windows)` 将会有相同的输出，而 `g 100 0` 则与它们不同。

`rnd` 对象的类型为 `random_t`，你可以建立一个新的随机值生成对象，不过通常你不需要这么做。

该对象有许多有用的成员函数，下面是一些例子：

调用	含义
<code>rnd.next(4)</code>	等概率生成一个 范围内的整数
<code>rnd.next(4, 100)</code>	等概率生成一个 范围内的整数

调用	含义
<code>rnd.next(10.0)</code>	等概率生成一个 范围内的浮点数
<code>rnd.next("one two three")</code>	等概率从 <code>one, two, three</code> 三个串中返回一个
<code>rnd.wnext(4, t)</code>	<code>wnext()</code> 是一个生成不等分布（具有偏移期望）的函数， 表示调用 <code>next()</code> 的次数，并取生成值的最大值。例如 <code>rnd.wnext(3, 1)</code> 等同于 <code>max({rnd.next(3), rnd.next(3)})</code> ； <code>rnd.wnext(4, 2)</code> 等同于 <code>max({rnd.next(4), rnd.next(4), rnd.next(4)})</code> 。如果 <code>t</code> ，则为调用 <code>t</code> 次，取最小值；如果 <code>t</code> ，等同于 <code>next()</code> 。
<code>rnd.any(container)</code>	等概率返回一个具有随机访问迭代器（如 <code>std::vector</code> 和 <code>std::string</code> ）的容器内的某一元素的引用

附：关于 `rnd.wnext(i, t)` 的形式化定义：

另外，不要使用 `std::random_shuffle()`，请使用 `Testlib` 中的 `shuffle()`，它同样接受一对迭代器。它使用 `rnd` 来打乱序列，即满足如上「好的 generator」的要求。

示例：生成一棵树

下面是生成一棵树的主要代码，它接受两个参数——顶点数和伸展度。例如，当 `n=1` 时，可能会生成链；当 `n=2` 时，可能会生成菊花。

```

1 #define forn(i, n) for (int i = 0; i < int(n); i++)
2
3 register Gen(argc, argv, 1);
4
5 int n = atoi(argv[1]);
6 int t = atoi(argv[2]);
7
8 vector<int> p(n);
9
10 /* 为节点 1..n-1 设置父亲 */
11 forn(i, n) if (i > 0) p[i] = rnd.wnext(i, t);
12
13 printf("%d\n", n);
14
15 /* 打乱节点 1..n-1 */
16 vector<int> perm(n);
17 forn(i, n) perm[i] = i;
18 shuffle(perm.begin() + 1, perm.end());
19
20 /* 根据打乱的节点顺序加边 */
21 vector<pair<int, int>> edges;
22 for (int i = 1; i < n; i++)
23     if (rnd.next(2))
24         edges.push_back(make_pair(perm[i], perm[p[i]]));
25     else
26         edges.push_back(make_pair(perm[p[i]], perm[i]));
27
28 /* 打乱边 */
29 shuffle(edges.begin(), edges.end());
30
31 for (int i = 0; i + 1 < n; i++)
32     printf("%d %d\n", edges[i].first + 1, edges[i].second + 1);

```

一次性生成多组数据

跟不使用 Testlib 编写的时候一样，每次输出前重定向输出流就好，不过 Testlib 提供了一个辅助函数 `startTest(test_index)`，它帮助你重定向输出流到 `test_index` 文件。

一些注意事项

- 严格遵循题目的格式要求，如空格和换行，注意文件的末尾应有一个换行。
- 对于大数据首选 `printf` 而非 `cout`，以提高性能。（不建议在使用 Testlib 时关闭流同步）
- 不使用 UB（Undefined Behavior，未定义行为），如本文开头的那个示例，输出如果写成 `cout << rnd.next(1, n) << " " << rnd.next(1, n) << endl;`，则 `rnd.next()` 的调用顺序没有定义。

新特性：解析命令行参数

在之前，我们通常使用类似 `int n = atoi(argv[3]);` 的代码，但是这样并不好。有以下几点原因：

- 不存在第三个命令行参数的时候是不安全的；
- 第三个命令行参数可能不是有效的 32 位整数。

现在，你可以这样写：`int n = opt<int>(3)`。与此同时，你也可以使用 `int64_t m = opt<int64_t>(1);`，`bool t = opt<bool>(2);` 和 `string s = opt(4);` 等。

另外，testlib 同时也支持命名参数。如果有很多参数，这样 `g 10 20000 a true` 的可读性就会比 `g -n10 -m200000 -t=a -increment` 差。

在这种情况下，现在你可以在 generator 中使用以下代码：

```
1 int n = opt<int>("n");
2 long long n = opt<long long>("m");
3 string t = opt("t");
4 bool increment = opt<bool>("increment");
```

你可以自由地混合使用按下标和按名称读取参数的方式。

支持的用于编写命名参数的方案有以下几种：

- `--key=value` 或 `-key=value`；
- `--key value` 或 `-key value` ——如果 `value` 不是新参数的开头（不以连字符 - 开头或一个/两个连字符后没有跟随字母）；
- `--k12345` 或 `-k12345` ——如果 `key k` 是一个字母，且后面是一个数字；
- `-prop` 或 `--prop` ——启用 `bool` 属性。

下面是一些例子：

```
1 g1 -n1
2 g2 --len=4 --s=oops
3 g3 -inc -shuffle -n=5
4 g4 --length 5 --total 21 -ord
```

更多示例

可以在 [GitHub](#) 中找到。

本文主要翻译自 [Генераторы на testlib.h - Codeforces](#)。新特性翻译自 [Testlib: Opts—parsing command line options](#)。testlib.h 的 GitHub 存储库为 [MikeMirzayanov/testlib](#)。

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [CCXXI](#), [countercurrent-time](#), [cubercsl](#), [lr1d](#), [NachtgeistW](#), [ouuan](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用